QUESTION 1 – CAMPUS DELIVERY ROBOT: GREEDY BEST-FIRST SEARCH

(a) Search strategy

The robot uses Greedy Best-First Search. At each junction it chooses the unvisited neighbouring location with the smallest heuristic h(n), where h(n) is the straight-line distance to the Auditorium. It considers only h(n), not the distance already travelled.

(b) Decision trace

Current location	Unvisited candidates and h(n)	Choice
Gate	Library = 330 m; Canteen = 240 m	Canteen (240 m)
Canteen	Hostel = 160 m; Admin = 110 m (Gate already visited)	Admin (110 m)
Admin	Auditorium = 0 m (Canteen already visited)	Auditorium

(c) Route and distance

Route: Gate → Canteen → Admin → Auditorium.

Total distance = 200 + 260 + 120 = 580 metres.

This route is not guaranteed to be the shortest because Greedy Best-First Search ignores the distance already travelled. A shorter route is Gate → Canteen → Hostel → Auditorium = 200 + 100 + 180 = 480 metres. Therefore, the greedy route is 100 metres longer.

Python program

```
import heapq

graph = {
    "Gate": {"Library": 120, "Canteen": 200},
    "Library": {"Gate": 120, "Lab": 150, "Hostel": 300},
    "Canteen": {"Gate": 200, "Hostel": 100, "Admin": 260},
    "Lab": {"Library": 150, "Auditorium": 350},
    "Hostel": {"Library": 300, "Canteen": 100, "Auditorium": 180},
    "Admin": {"Canteen": 260, "Auditorium": 120},
    "Auditorium": {"Lab": 350, "Hostel": 180, "Admin": 120}
}
h = {"Gate": 400, "Library": 330, "Canteen": 240,
     "Lab": 280, "Hostel": 160, "Admin": 110, "Auditorium": 0}

def greedy_best_first(start, goal):
    frontier = [(h[start], start, [start], 0)]
    visited = set()
    while frontier:
        _, node, path, distance = heapq.heappop(frontier)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, distance
        for nxt, cost in graph[node].items():
            if nxt not in visited:
                heapq.heappush(frontier, (h[nxt], nxt, path + [nxt], distance +
    return None, float("inf")

path, distance = greedy_best_first("Gate", "Auditorium")
print("Route:", " -> ".join(path))
print("Distance:", distance, "metres")
```

Output:

```
Route: Gate -> Canteen -> Admin -> Auditorium
Distance: 580 metres
>>>
```

QUESTION 2 – A* SEARCH

(a) Strategy and guarantee

The new strategy is A* Search. It evaluates each candidate using f(n) = g(n) + h(n), where g(n) is the distance from the Gate so far and h(n) is the estimated straight-line distance to the Auditorium. A* finds an optimal route when the heuristic is admissible (never overestimates the true remaining cost) and the search uses the usual graph-search handling for path costs.

(b) A* trace

Step	Candidate	g(n) travelled (m)	h(n) (m)	f(n)=g+h (m)	Decision
1	Library	120	330	450	Frontier
1	Canteen	200	240	440	Expand first (lowest f)
2	Hostel via Canteen	300	160	460	Frontier
2	Admin via Canteen	460	110	570	Frontier
3	Lab via Library	270	280	550	Frontier after Library expanded
3	Hostel via Library	420	160	580	Discard: worse than g=300 route
4	Auditorium via Hostel	480	0	480	Expand goal; stop

Expansion order: Gate, Canteen, Library, Hostel, Auditorium. The resulting route is Gate → Canteen → Hostel → Auditorium.

(c) Heuristic safety and comparison

The heuristic values are safe for these locations because they do not exceed the true shortest remaining road distance:

Location	h(n) given (m)	True shortest remaining distance (m)	Admissible?
Gate	400	480	Yes
Library	330	480	Yes
Canteen	240	280	Yes
Hostel	160	180	Yes

A* gives a route of 480 m, compared with the 580 m greedy route in Question 1. The A* route is 100 m shorter.

Python program

```
import heapq

graph = {
    "Gate": {"Library": 120, "Canteen": 200},
    "Library": {"Gate": 120, "Lab": 150, "Hostel": 300},
    "Canteen": {"Gate": 200, "Hostel": 100, "Admin": 260},
    "Lab": {"Library": 150, "Auditorium": 350},
    "Hostel": {"Library": 300, "Canteen": 100, "Auditorium": 180},
    "Admin": {"Canteen": 260, "Auditorium": 120},
    "Auditorium": {"Lab": 350, "Hostel": 180, "Admin": 120}
}
h = {"Gate": 400, "Library": 330, "Canteen": 240,
     "Lab": 280, "Hostel": 160, "Admin": 110, "Auditorium": 0}
```

```
def astar(start, goal):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if g != best_g.get(node):
            continue
        if node == goal:
            return path, g
        for nxt, cost in graph[node].items():
            new_g = g + cost
            if new_g < best_g.get(nxt, float("inf")):
                best_g[nxt] = new_g
                heapq.heappush(pq, (new_g + h[nxt], new_g, nxt, path + [nxt]))
    return None, float("inf")
```

```
path, distance = astar("Gate", "Auditorium")
print("Route:", " -> ".join(path))
print("Shortest distance:", distance, "metres")
```

Output:

```
Route: Gate -> Canteen -> Hostel -> Auditorium
Shortest distance: 480 metres
>>>
```

QUESTION 3 – CONSTRAINT SATISFACTION PROBLEM (CSP)

(a) Variables, domains and constraints

This is a Constraint Satisfaction Problem. Variables: W1, W2, W3, W4, W5. Initial domains: {Morning, Afternoon, Evening} for each workshop, except W3 = {Afternoon, Evening} because its trainer is unavailable in the Morning, and W4 = {Morning, Afternoon} because its hall is unavailable in the Evening.

Binary constraints (conflicting workshops must have different slots): W1 ≠ W2, W1 ≠ W3, W2 ≠ W3, W2 ≠ W4, W3 ≠ W5, W4 ≠ W5.

(b) MRV timetable construction and domain reduction

Step	Assignment	Remaining domains after propagation
Initial	None	W1={M,A,E}; W2={M,A,E}; W3={A,E}; W4={M,A}; W5={M,A,E}
1	W3 = Afternoon	W1={M,E}; W2={M,E}; W4={M,A}; W5={M,E}
2	W2 = Morning	W1={E}; W4={A}; W5={M,E}
3	W4 = Afternoon	W1={E}; W5={M,E}
4	W1 = Evening	W5={M,E}
5	W5 = Morning	All workshops assigned

Valid timetable: Morning – W2 and W5; Afternoon – W3 and W4; Evening – W1. Every conflicting pair has different slots, and the availability restrictions are satisfied.

(c) Why MRV helps

The Minimum Remaining Values (MRV) heuristic selects the workshop with the fewest legal slots first. This exposes highly constrained choices early and can detect a dead end before many other variables are assigned. If any workshop has no slot left, the current partial assignment is inconsistent; backtrack to the most recent choice and try another available slot.

Python program

```
slots = ["Morning", "Afternoon", "Evening"]
neighbors = {
    "W1": {"W2", "W3"},
    "W2": {"W1", "W3", "W4"},
    "W3": {"W1", "W2", "W5"},
    "W4": {"W2", "W5"},
    "W5": {"W3", "W4"}
}
domains = {
    "W1": ["Morning", "Afternoon", "Evening"],
    "W2": ["Morning", "Afternoon", "Evening"],
    "W3": ["Afternoon", "Evening"],
    "W4": ["Morning", "Afternoon"],
    "W5": ["Morning", "Afternoon", "Evening"]
}
def backtrack(assignment):
    if len(assignment) == len(domains):
        return assignment.copy()
    unassigned = [w for w in domains if w not in assignment]
    def key(w):
        legal = [s for s in domains[w]
                 if all(assignment.get(n) != s for n in neighbors[w])]
        degree = sum(n not in assignment for n in neighbors[w])
        return (len(legal), -degree, int(w[1:]))
    var = min(unassigned, key=key)
    legal_values = [s for s in domains[var]
                    if all(assignment.get(n) != s for n in neighbors[var])]
    for value in legal_values:
        assignment[var] = value
        result = backtrack(assignment)
        if result is not None:
            return result
        del assignment[var]
    return None
solution = backtrack({})
print(solution)
```

Output:

```
{'W3': 'Afternoon', 'W2': 'Morning', 'W4': 'Afternoon', 'W1': 'Evening', 'W5': 'Morning'}
>>>
```

QUESTION 4 – MINIMAX

(a) Technique and assumption

The bot should use the Minimax algorithm. It assumes Player O is rational and will choose the reply that minimizes X's final score, while X chooses moves that maximize the score.

(b) Values from leaves to root

Node / leaf pair	X's final move value (MAX)	O reply value (MIN)
Leaves 4, 7	max(4,7) = 7	—
Leaves 8, 2	max(8,2) = 8	—
O replies under X's first move	—	min(7,8) = 7
Leaves 3, 5	max(3,5) = 5	—
Leaves 6, 9	max(6,9) = 9	—
O replies under X's second move	—	min(5,9) = 5
Starting node (X chooses)	—	max(7,5) = 7

(c) Best move and line of play

X chooses its first move (the left branch), guaranteeing a score of 7. O then chooses the reply with value 7 rather than 8. X chooses the final leaf scored 7. Optimal-play score: 7.

Python program

```
def minimax(node, depth, maximizing):
    if depth == 0:
        return node
    if maximizing:
        return max(minimax(child, depth - 1, False) for child in node)
    return min(minimax(child, depth - 1, True) for child in node)
tree = [
    [[4, 7], [8, 2]],
    [[3, 5], [6, 9]]
]
values = [minimax(branch, 2, False) for branch in tree]
best_score = max(values)
best_move = values.index(best_score) + 1
print("Values of X's first moves:", values)
print("Best first move: X move", best_move)
print("Guaranteed score:", best_score)
```

Output:

```
-------------------------------------------------
Values of X's first moves: [7, 5]
Best first move: X move 1
Guaranteed score: 7
>>>
```

QUESTION 5 – ALPHA-BETA PRUNING

(a) Evaluation from left to right

The engine is MAX and the opponent is MIN. Alpha (α) is the best value MAX can guarantee so far; beta (β) is the best upper bound MIN can guarantee. Prune remaining children whenever $\beta \leq \alpha$.

Move	Values examined	α before / after	β at MIN node	Result
M1	6, 9, 7	$-\infty \rightarrow 6$	6	M1 value = min(6,9,7)=6
M2	4; skip 8 and 10	α=6	After first value β=4 ≤ α=6	Prune 8, 10; M2 cannot beat 6
M3	7, 5; skip 12	α=6	After second value β=5 ≤ α=6	Prune 12; M3 cannot beat 6

(b) Number scored and decision

Five of the nine leaf positions are scored: all three under M1, one under M2, and two under M3. The selected move is M1 with value 6. Full minimax gives M1=min(6,9,7)=6, M2=min(4,8,10)=4, M3=min(7,5,12)=5, so the result is the same as examining all nine positions.

(c) Move ordering and evaluation function

Move ordering affects how early a strong alpha or low beta bound is found. Examining promising moves first usually enables more pruning; poor ordering may cause fewer branches to be skipped. An evaluation function estimates the desirability of a non-terminal position when the search cannot reach the actual end of the game.

Python program

```
def alphabeta(values, alpha, beta, move_name):
    best = float("inf")
    examined = []
    skipped = []
    for i, value in enumerate(values):
        examined.append(value)
        best = min(best, value)
        beta = min(beta, best)
        if beta <= alpha:
            skipped.extend(values[i + 1:])
            break
    return best, examined, skipped
moves = {"M1": [6, 9, 7], "M2": [4, 8, 10], "M3": [7, 5, 12]}
alpha = float("-inf")
results = {}
total_scored = 0

for name, values in moves.items():
    value, examined, skipped = alphabeta(values, alpha, float("inf"), name)
    results[name] = value
    total_scored += len(examined)
    alpha = max(alpha, value)
    print(name, "value =", value, "examined =", examined, "skipped =", skipped,
          "alpha =", alpha)
best_move = max(results, key=results.get)
print("Positions scored:", total_scored, "of 9")
print("Chosen move:", best_move, "with value", results[best_move])
```

Output:

```
M1 value = 6 examined = [6, 9, 7] skipped = [] alpha = 6
M2 value = 4 examined = [4] skipped = [8, 10] alpha = 6
M3 value = 5 examined = [7, 5] skipped = [12] alpha = 6
Positions scored: 6 of 9
Chosen move: M1 with value 6
>>>
```

Conclusion

Greedy Best-First Search chooses by heuristic alone and can return a non-optimal route. A* combines path cost and heuristic. CSPs represent scheduling using variables, domains and constraints, with MRV reducing search. Minimax chooses a move against an optimal opponent, and Alpha-Beta Pruning returns the same minimax decision while avoiding evaluation of branches that cannot affect the result.